

CS 650 – Full Stack Application Develop

8-1 Activity: Security Monitoring in a Full Stack Application

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

August 25, 2026

Current Monitoring Approach

The original RoadFix application had limited operational visibility. It printed server-startup and database-seeding messages to the console, and `middleware.ts` contained a structured request logger and a rate-limit warning handler. However, the request logger was not connected to the Express application, and the dedicated `logSecurityEvent` function in `logger.ts` was only a stub. As a result, authentication outcomes, rejected authorization attempts, and petition-record changes did not create an audit trail. Security logging could be added at the points where the application already makes security decisions. These points include the registration, login, and logout routes; the `requireAuth` and `requireOrganizer` middleware; and the POST and DELETE signature routes. The existing request middleware could also be enabled after the rate limiter, so each completed request records its method, path, response status, and duration.

Monitoring Approaches and Tools

Plain console messages are convenient for debugging but are difficult to search reliably because their wording and fields are inconsistent. Structured JSON application logs are more useful for security monitoring because every entry can use the same timestamp, event, severity, and context fields. A production team could forward these JSON records to a centralized log platform or security information and event management system, where searches, dashboards, retention rules, and alerts could identify repeated failures across servers. For this activity, local structured JSON output provides a practical implementation that can later be collected by such a platform.

Security Monitoring Implementation

I completed `logSecurityEvent` so it creates an ISO 8601 timestamp, event name, severity, and event-specific context, serializes the object with `JSON.stringify`, and writes one JSON record per

line. I also enabled `requestLogger()` in the Express middleware chain. Information-level events record expected successful actions; warning-level events identify failed authentication, rejected access, rate limiting, and deletion of a petition record.

The authentication logs cover registration, login success, login failure, and logout. Authorization logs cover unauthenticated requests to protected routes and authenticated residents attempting organizer-only routes. Data-change logs cover signature creation and deletion. Context fields include only what is needed to investigate the event: user ID and email for attributable account activity, role for a forbidden request, method and path for rejected access, signature ID for record changes, and a generic reason for failed login. Passwords, session cookies, password hashes, petition comments, names, and request bodies are not logged.

```
21     severity: Severity,  
22     context: Record<string, unknown> = {},  
23 ): void {  
24     const entry = {  
25         timestamp: new Date().toISOString(),  
26         event,  
27         severity,  
28         ...context,  
29     };  
30  
31     console.log(JSON.stringify(entry));  
32 }
```

Figure 1. Structured security logger implemented in `backend/logger.ts`.

Figure 1 shows the shared organization used by every security event. A consistent schema makes the output machine-readable and allows a team to filter by event or severity while retaining the additional fields needed for a specific investigation.

Logged Event Organization

auth:register (info): Records the newly created user ID and email address.

auth:login and auth:logout (info): Record the authenticated account entering or ending a session; login includes user ID and email, while logout retains only the user ID.

auth:login_failed (warn): Records the attempted email and a generic failure reason, never the submitted password.

auth:unauthorized and auth:forbidden (warn): Record the method and protected path; forbidden events also identify the authenticated user ID and role.

data:signature_created and data:signature_deleted (info/warn): Record the acting user ID, account email, and affected signature ID without capturing the petition comment.

http:request (info/warn): Records method, route, status, and duration for completed requests; the rate-limit handler uses the same event with a warning severity and rate_limited reason.

```

function requireAuth(req: Request, res: Response, next: NextFunction): void {
  if (!req.session.userId) {
    logSecurityEvent("auth:unauthorized", "warn", {
      method: req.method,
      path: req.path,
    });

    res.status(401).json({ error: "Authentication required" });
    return;
  }
  next();
}

function requireOrganizer() {
  return async (req: Request, res: Response, next: NextFunction) => {
    if (!req.session.userId) {
      logSecurityEvent("auth:unauthorized", "warn", {
        method: req.method,
        path: req.path,
      });

      res.status(401).json({ error: "Authentication required" });
      return;
    }
    const user = await getUserById(db, req.session.userId);
    if (!user || user.role !== "organizer") {
      logSecurityEvent("auth:forbidden", "warn", {
        userId: req.session.userId,
        role: user?.role ?? "unknown",
        method: req.method,
        path: req.path,
      });

      res.status(403).json({ error: "Organizer access required" });
      return;
    }
    next();
  }
}

```

Figure 2. Authorization middleware and selected authentication and petition-event calls in backend/server.ts.

Figure 2 shows that security logging occurs at the actual decision and modification points rather than being inferred only from a generic HTTP status. For example, a 403 response is paired with the resident's user ID and role, while a deletion identifies both the organizer and the affected signature record.

Verification Procedure and Results

I reset and seeded the SQLite database, started the backend, and then used HTTP requests with separate cookie files to reproduce the monitored events. I performed the following checks:

1. Requested GET /api/signatures without a session to trigger auth:unauthorized and HTTP 401.
2. Submitted James Chen's email with an incorrect password to trigger auth:login_failed and HTTP 401.
3. Logged in as resident Lisa Thompson, requested the organizer-only signature list to trigger auth:forbidden and HTTP 403, and created a petition signature to trigger data:signature_created and HTTP 201.
4. Logged Lisa out to trigger auth:logout and HTTP 204.
5. Logged in as organizer Maria Santos and deleted the newly created signature to trigger data:signature_deleted and HTTP 204.

All tested routes returned the expected HTTP status, and the console displayed the corresponding JSON event. The generic http:request records also confirmed the route, status, and elapsed time. This pairing demonstrates that both the security-specific instrumentation and the request-level monitoring were functioning.

```

added 131 packages, and audited 132 packages in 2s

40 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

npm notice
npm notice New major version of npm available! 10.9.7 -> 12.0.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.0.2
npm notice To update run: npm install -g npm@12.0.2
npm notice
malgorzatadebska@Malgorzatas-iMac backend % npm run seed

> roadfix-backend@1.0.0 seed
> node seed.ts

Seeded 5 users
Seeded 1 proposal
Seeded 3 signatures
Done!
malgorzatadebska@Malgorzatas-iMac backend % npm run start

> roadfix-backend@1.0.0 start
> node server.ts

RoadFix server running on http://localhost:3000
{"timestamp":"2026-08-27T14:01:20.153Z","event":{"auth:unauthorized","severity":"warn","method":"GET","path":"/api/signatures"}
{"timestamp":"2026-08-27T14:01:20.156Z","event":{"http:request","severity":"info","method":"GET","path":"/api/signatures","status":401,"durationMs":5}
{"timestamp":"2026-08-27T14:01:38.044Z","event":{"auth:login_failed","severity":"warn","email":"jamesoakdale.example.com","reason":"Invalid email or password"}
{"timestamp":"2026-08-27T14:01:38.045Z","event":{"http:request","severity":"info","method":"POST","path":"/api/auth/login","status":401,"durationMs":183}

```

Figure 3. Actual terminal output showing database seeding, server startup, unauthorized access, and a failed login.

[illegible]

Figure 4. Actual server output showing successful resident login, forbidden organizer-route access, and signature creation. The wide terminal capture is stacked for readability.

The output records Lisa Thompson's successful login as user 5, her resident role during the rejected organizer-only request, HTTP 403 for the protected signature list, and creation of signature 4 with HTTP 201.

```

Roadfix server running on http://localhost:3000
{"timestamp":"2020-08-27T14:01:18.153Z","event":"auth:unauthorized","severity":"warn","method":"GET","path":"/api/signatures","status":401,"durationMs":5}
{"timestamp":"2020-08-27T14:01:18.154Z","event":"http:request","severity":"info","method":"GET","path":"/api/signatures","status":401,"durationMs":5}
{"timestamp":"2020-08-27T14:01:18.844Z","event":"auth:login_failed","severity":"warn","email":"jane@okadale.example.com","reason":"invalid email or password"}
{"timestamp":"2020-08-27T14:01:18.845Z","event":"http:request","severity":"info","method":"POST","path":"/api/auth/login","status":401,"durationMs":183}
{"timestamp":"2020-08-27T14:01:11.584Z","event":"http:request","severity":"info","method":"POST","path":"/api/auth/login","status":200,"durationMs":97}
{"timestamp":"2020-08-27T14:01:11.584Z","event":"auth:login","severity":"info","method":"POST","path":"/api/auth/login","status":200,"durationMs":97}
{"timestamp":"2020-08-27T14:01:12.384Z","event":"auth:login","severity":"warn","userId":5,"email":"lisa@okadale.example.com"}
{"timestamp":"2020-08-27T14:01:12.384Z","event":"http:request","severity":"info","method":"GET","path":"/api/signatures","status":401,"durationMs":2}
{"timestamp":"2020-08-27T14:01:14.475Z","event":"data:signature_created","severity":"info","userId":5,"email":"lisa@okadale.example.com","signatureId":4}
{"timestamp":"2020-08-27T14:01:14.474Z","event":"http:request","severity":"info","method":"POST","path":"/api/signatures","status":201,"durationMs":77}
{"timestamp":"2020-08-27T14:01:17.885Z","event":"http:request","severity":"info","method":"POST","path":"/api/auth/logout","status":204,"durationMs":23}
{"timestamp":"2020-08-27T14:01:17.847Z","event":"auth:logout","severity":"info","userId":5}
{"timestamp":"2020-08-27T14:01:17.847Z","event":"http:request","severity":"info","method":"POST","path":"/api/auth/login","status":200,"durationMs":94}
{"timestamp":"2020-08-27T14:01:17.848Z","event":"data:signature_deleted","severity":"warn","userId":3,"email":"maria@okadale.example.com","signatureId":4}
{"timestamp":"2020-08-27T14:01:17.887Z","event":"http:request","severity":"info","method":"DELETE","path":"/api/signatures/4","status":204,"durationMs":15}

```

Figure 5. Actual server output showing resident logout, organizer login, and deletion of signature 4. The wide terminal capture is stacked for readability.

The final security sequence records Lisa’s logout, Maria Santos’s successful organizer login as user 1, data:signature_deleted for signature 4, and the matching HTTP 204 deletion response.

```

malgorzatadebska@Malgorzatas-iMac backend % cd ../frontend
malgorzatadebska@Malgorzatas-iMac frontend % npm install

added 43 packages, and audited 44 packages in 25s

12 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
malgorzatadebska@Malgorzatas-iMac frontend % npm run build

> roadfix-frontend@1.0.0 build
> tsc -b && vite build

vite v8.2.2 building client environment for production...
✓ 28 modules transformed.
computing gzip size...
dist/index.html                0.42 kB | gzip: 0.29 kB
dist/assets/index-CULb6X2d.css 10.63 kB | gzip: 2.89 kB
dist/assets/index-BYvu7BU6.js 239.99 kB | gzip: 75.67 kB

✓ built in 354ms
malgorzatadebska@Malgorzatas-iMac frontend %

```

Figure 6. Actual terminal output confirming the frontend production build completed successfully.

Effectiveness of the Monitoring Approach

The logs would help a team identify repeated failed logins against one or several accounts, unauthenticated probing of protected endpoints, residents attempting to reach organizer

functions, and unexpected creation or deletion of petition signatures. During an incident, timestamps, user IDs, emails, routes, response statuses, and signature IDs could reconstruct the sequence of actions and distinguish a normal error from a suspicious pattern. Warning-level deletion records also provide accountability for organizer actions that change the petition of evidence.

The current approach still has important gaps. It does not automatically aggregate repeated events, generate alerts, persist logs outside standard output, protect logs from alteration, or correlate activity across multiple application instances. It also does not record a request or correlation of ID, client IP address, user agent, session identifier, or the prior state of a deleted record. Therefore, it would not independently detect a distributed attack, session theft, successful use of stolen credentials, database changes made outside the application, malicious changes that use an authorized organizer account, or an attacker who compromises and deletes the local logs. Input-validation failures and unexpected server exceptions also need dedicated security events if the application is expanded.

Privacy Implications and Data Minimization

The implementation captures email addresses and internal user IDs for login and petition events. It also records a user's role, the requested route, timestamps, response status, and petition signature IDs. Email addresses are personally identifiable for information, and a signature event can reveal a resident's participation in a neighborhood tax-assessment effort. If the logs were exposed, an unauthorized person could associate named accounts with petition activity and infer when residents used the application. The timing and route history could also reveal behavioral patterns.

Security visibility should be balanced with data minimization by collecting only fields tied to a defined detection or investigation purpose. Passwords, hashes, cookies, tokens, comments, names, full request bodies, and unnecessary query strings should remain excluded. In production, the team could replace plain emails with a stable keyed hash or rely primarily on internal user IDs, add IP addresses only when justified and truncate or hash them where practical, restrict log access by role, encrypt logs in transit and at rest, and separate security logs from general debugging output. A short, documented retention period should delete routine records when they are no longer needed while preserving incident evidence under an approved hold. Alert rules should use event counts and severity without broadly exposing raw personal fields.